

SALESTORM

HIGH-SCALE FLASH SALE SYSTEM

10,000 concurrent customers compete for 100 units — engineered for absolute zero overselling.

CONCURRENCY	INVENTORY	OVERSELLING
10,000	100 UNITS	0 STRICT

Made with GAMMA

System Requirements

Functional

Login, browsing, cart, flash-sale purchase, reservation, expiry, payment, orders, shipment tracking, notifications.

Non-functional

Scalability, availability, performance, consistency, reliability, security, fault tolerance, observability.

Success criteria

- Overselling = 0
- Duplicate payment = 0
- Successful reservations ≤ 100

Made with GAMMA

TOPOLOGY

High-Level System Architecture

Tiered edge isolation, decoupled domain microservices, and asynchronous persistence.

01

EDGE INGRESS

Users hit CDN / WAF for DDoS filtering, SSL termination, and static asset caching before reaching global Load Balancers.

02

ROUTING GATEWAY

API Gateway orchestrates token auth, client rate limiting, protocol translation, and routes calls to downstream domain services.

03

CORE DOMAINS

User, Product, Cart, Sale, Inventory / Reservation, Checkout, Payment, and Order microservices scale independently.

04

FULFILLMENT TIER

Asynchronous event stream triggers Shipment and Notification pipelines without blocking critical customer checkout path.

DATA BUS

MESSAGE QUEUE

Kafka event buffers decoupling order processing

MEMORY TIER

REDIS CACHE

Atomic reservation locks & rate limit keys

STORAGE

SQL DATABASE

ACID relational records with read replicas

TELEMETRY

APM & TRACING

Metrics, distributed traces, and log pipelines

Made with GAMMA

CORE INVARIANT ENGINE

Critical Challenge: 10,000 Customers vs 100 Units

CONCURRENT REQUESTS

10,000

INVENTORY / RESERVATION SERVICE

Atomic, concurrency-controlled reservation

Inventory is the consistency boundary: check and reserve atomically.

Only 100 successes → remaining requests are out of stock.

INVENTORY STATE MACHINE

Idempotency: blocks duplicate reservations.

Expiry: releases stock when payment is not completed.

State path:
Available → Reserved → Payment Pending → Confirmed → Sold

Payment failed / timeout → release reservation.

OVERSELLING = 0.

Made with GAMMA

LIFECYCLE EXECUTION

Payment and Order Workflow

Asynchronous event-driven pipeline with transactional guarantees and auto-rollback.

01

1. CHECKOUT

User initiates with temporary reservation token.

02

2. GATEWAY

Payment Service calls gateway with idempotency key.

03

3. EVENT PUB

On success, publish event to durable message queue.

04

4. ORDER GEN

Order Service consumes event and commits records.

05

5. FULFILLMENT

Shipment booked and user notification dispatched.

FAILURE COMPENSATION PATH

If payment fails or timer expires, the reservation key is automatically unlocked in Redis, instantly returning units to Available for waitlisted users.

ORDER SERVICE RESILIENCE

If Order Service is temporarily degraded, payment events persist safely in Message Queue with exponential backoff retries until order creation succeeds.

Made with GAMMA

SYSTEM ROBUSTNESS

Scalability, Reliability & Fault Tolerance

SCALE & TRAFFIC SHAPE

Traffic shape: normal traffic ~10K req/s; flash-sale traffic up to ~500K req/s.

- **Scale:** CDN / WAF, load balancing, horizontal services
- Redis caching, message queues, database scaling, rate limiting

RELIABILITY & OBSERVABILITY

- **Reliability:** idempotency, retries, circuit breaker, reservation timeout, DLQ, database failover, monitoring and alerts.
- **Observability:** metrics + logs + distributed tracing + alerts.

Made with GAMMA

Conclusion

10,000

CONCURRENT CUSTOMERS

100

AVAILABLE UNITS

0

OVERSELLING

DESIGN OUTCOMES

- Concurrency-controlled inventory reservation
- Idempotent payments and reservation expiry
- Message queue for reliable asynchronous processing
- Horizontal scaling, fault tolerance, security, and observability

SALESTORM

SALESTORM provides a scalable, reliable, consistency-focused architecture for high-demand flash sales.

Thank You.

Made with GAMMA